

GUÍA RESUMEN — Debugging Flutter

Resumen integral del debugging y la optimización en Flutter sin volver a leer el módulo completo. Hilo conductor: Modos de ejecución → launch.json y breakpoints → Debug Console → DevTools (Inspector, Performance, Memory, CPU, Network, Logging, App Size) → Rendimiento (rebuilds, memory leaks, rendering) → Debugging asíncrono → Workflow por tipo de bug.

0 Modos de ejecución y fundamentos

DEBUG • PROFILE • RELEASE

Modo	Para qué	Optimizaciones
debug	Depurar con breakpoints, hot reload	Ninguna (lento)
profile	Medir rendimiento (siempre aquí)	Mayoría habilitadas
release	Publicar, verificar builds	Todas

Regla de oro: para debugging usa `debug` ; para medir performance usa `profile` — nunca midas en debug (el JIT distorsiona todo). Flag clave: `--dart-define` para config por entorno y `--start-paused` para esperar al debugger.

1 launch.json y breakpoints

F5 • LOGPOINTS • CONDITIONALS

`launch.json` define configuraciones reutilizables: nombre, programa, `--dart-define` , device, args. Con una configuración `flutter attach` puedes conectar el debugger a un app ya corriendo (sin `--debug-port`).

Breakpoint	Uso
Normal	Pausa la ejecución en una línea
Conditional	Pausa solo si la condición es true
Logpoint	Imprime en Debug Console sin pausar (reemplaza <code>print()</code>)
Function	Pausa al entrar a una función

Regla: usa logpoints en lugar de `print()` para debugging temporal — no contaminan el código y se pueden dejar activos sin pausar la app.

2 Inspección de datos y Debug Console

VARIABLES • WATCH • REPL

- VARIABLES:** valores de las variables locales durante el pause.
- WATCH:** expresiones que se reevalúan en cada pause (ej. `user.name` , `items.length`).
- Debug Console = REPL:** ejecuta código Dart en el contexto actual de la sesión (crear objetos, llamar métodos).
- Multi-target:** launch.json con configuraciones por device (android, ios, chrome) y debugging remoto por red.

Flujo típico: breakpoint → inspeccionar VARIABLES → evaluar expresiones en Debug Console → avanzar con F10/F11/F5 (Step Over/Into/Continue).

DevTools es la suite de profiling y diagnóstico de Flutter. Se abre con `dart devtools <vm-service-uri>` (URI posicional del VM Service) o desde VS Code con el comando `Dart: Open DevTools`.

Vista	Para qué
Flutter Inspector	Widget tree, layout, constraints
Performance	Frames, jank, flame chart
Memory	Heap, leaks, GC, snapshots
CPU Profiler	Flame chart por sampling
Network	Requests HTTP y WebSocket
Debugger	Breakpoints sincronizados con VS Code
Logging	Logs de <code>package:logging</code> y <code>print</code>
App Size	Análisis del tamaño del build

Configuración: `dart.devToolsTheme` y `dart.devToolsBrowser` en settings de VS Code. El Debugger de DevTools comparte el mismo VM Service que VS Code: los breakpoints se sincronizan automáticamente.

4 Flutter Inspector y Performance

Flutter Inspector: explora el widget tree, selecciona widgets, inspecciona propiedades y constraints. Usa **Select Widget Mode** para encontrar dónde se ve mal un widget. MediaQuery responsive: `MediaQuery.sizeOf(context)`.

Performance View: graba el timeline y muestra cada frame. Un frame debe completarse en **<16.6ms (60fps)**.

Señal	Significado
Frame rojo	UI thread lento (build/layout)
Frame morado	Raster thread lento (paint/compositing)
Build time > 8ms	Rebuilds excesivos en listas
Enhance Tracing	Widget rebuilds por elemento

Diagnóstico: graba 5-10 segundos de interacción → identifica el frame más lento → analiza el flame chart → clasifica (build vs raster).

5 CPU Profiler y Memory View

CPU Profiler: muestrea la CPU (sample rate por defecto) y genera un flame chart: qué función consume más tiempo de ejecución.

Memoria	Uso
Take Heap Snapshot	Estado de la memoria en un momento
Diff Snapshot	Compara dos momentos → detecta clases que crecen
GC	Colecciones de basura del isolate

Detectar leaks: snapshot base → navegar 5-10 veces a la pantalla sospechosa → diff snapshot → buscar `StreamSubscription`, `TextEditingController`, `Timer`, `AnimationController` que crecen → **fix:** cancelar/cerrar en `dispose()`.

6 Network, Logging y Debugger View

- Network:** lista requests HTTP/WebSocket, ver body, headers y status code. Útil para APIs (Supabase, dio). Copia como cURL para probar fuera.
- Logging:** interpreta niveles de `package:logging` (FINE, INFO, WARNING, SEVERE, SHOUT) + prints. Filtros por nivel mínimo, texto y tag.
- Debugger View:** alternativa de DevTools al debugging de VS Code: pausa, variables, call stack y **logpoints**, todo en el navegador.

Recomendación: Network View para debugging de API, Debug Console para lógica, Logging para trazas largas. Para debugging extensivo, VS Code es más rápido en el día a día.

7 App Size: reducir el tamaño del build

ANALYZE-SIZE · LÍMITES DE STORES

```
flutter build apk --analyze-size
flutter build appbundle --analyze-size
flutter build ios --analyze-size
```

- **Google Play:** límite 200 MB para AAB.
- **App Store:** 2 GB binario; >200 MB no descargable por datos móviles.
- **Deferred imports** para código pesado (solo web).
- Mayores culpables: assets sin comprimir, imágenes enormes, dependencias no usadas (usa `flutter pub deps` y remueve).

Pipeline: `--analyze-size` genera un reporte visual en Chrome DevTools App Size → identificar el paquete/asset dominante → optimizar (imágenes, `shrink`, `--split-debug-info`).

8 Rendimiento: rebuilds, const y RepaintBoundary

BUILD · PAINT · 60FPS

Cada frame tiene 2 fases: **build/layout** (UI thread) y **paint/raster** (raster thread). El objetivo: mantener ambas < 8ms.

Técnica	Efecto
<code>const</code>	Widgets estáticos no se reconstruyen (reducción del 50-80% en UIs estáticas)
<code>ListView.builder</code>	Solo construye items visibles
<code>BlocSelector</code> / <code>buildWhen</code>	Rebuilds selectivos del subtree
<code>RepaintBoundary</code>	Aísla el repintado (animaciones en listas)
<code>shouldRepaint</code>	<code>CustomPainter</code> solo repinta si cambia
Extraer widgets	Cada widget se reconstruye de forma independiente

Regla: solo optimiza donde tengas un problema medible en profile mode. Agregar `RepaintBoundary` incorrectamente puede *empeorar* el rendimiento.

9 Memory leaks y rendering complejo

DISPOSE · SLIVERS · IMÁGENES

Patrones de leak (y su fix)

- `StreamSubscription` → guardar referencia y `cancel()` en `dispose()`.
- `TextEditingController` / `ScrollController` → `dispose()` (y `removeListener`).
- `Timer.periodic` → `cancel()` en `dispose()`.
- `setState` después de `await` → `if (!mounted) return`.

Rendering complejo

- `ListView.builder` vs `ListView`: lazily built siempre para >20 items.
- `CustomScrollView` + Slivers para layouts de scroll complejos; `SliverFixedExtentList` si el alto es fijo.
- Imágenes: `cacheWidth` / `cacheHeight` según el tamaño de display + `precacheImage`.
- Evita `Opacity` sobre subtrees grandes; usa `AnimatedOpacity` o `IgnorePointer`.

Memoria por imagen: 4000×3000 = 48 MB sin `cacheWidth`; con redimensión a 200×150 ≈ 120 KB. La diferencia es ~400×.

Prácticas VS Code (07): debugging de BLoC/Cubit, Supabase, GoRouter y UI + ejercicio integrador de pagos.

Prácticas DevTools (18): cinco ejercicios guiados + integrador para diagnosticar una app "enferma".

Prácticas de optimización (24): seis escenarios: lista congelada (`ListView.builder`), login parpadeante (`BlocSelector`), chat con leak (`dispose()`), dashboard pesado (secciones), imágenes lentas (`cacheWidth`), app que crece (controllers).

Cheatsheet	Clave
06 (VS Code)	F5/F9/F10/F11, logpoints, Run without Debugging <code>Ctrl+F5</code>
17 (DevTools)	Vista por escenario de debugging
23 (Optimización)	Problema → causa → solución → DevTools View; métricas saludables; mitos

Métricas saludables: build/raster < 4ms (excelente) · FPS 60 · memory growth 0 MB/min · app startup < 1s · profundidad del árbol < 15.

Los 5 bugs asíncronos: Future que nunca completa (falta try/catch) · setState sin `mounted` check · llamadas secuenciales innecesarias (usa `Future.wait`) · stream sin cancelar · `Completer` manual innecesario.

Patrón de carga segura: try/catch alrededor del await + estado de loading/error/data + timeout (`fetch().timeout(...)`) + trabajo pesado delegado a `compute()` para no bloquear el UI thread.

Tipo de bug	Herramienta principal
UI se ve mal	Flutter Inspector → Select Widget
Estado inconsistente	Prints + Bloc Inspector
App lenta	Performance View (frames >16ms)
API falla	Network View + cURL
Memory leak	Memory View → diff snapshots
Build error	Leer el error completo
Hot Reload no funciona	Hot Restart (no recarga initState/main/imports)

Framework general: REPRODUCIR → AISLAR → DIAGNOSTICAR → CORREGIR → VERIFICAR.

A Anexo · Índice del módulo (para profundizar)

Necesitas	Archivo
Modos debug/profile/release, componentes del debugger	01-fundamentos-debugging.md
launch.json, flutter attach, dart-define, args	02-configuracion-launch-json.md
Breakpoints conditional/function/logpoint, práctica con streams	03-breakpoints-avanzados.md
VARIABLES, WATCH, Debug Console como REPL	04-inspeccion-datos-consola.md
Multi-device, debugging remoto por red	05-multi-target-remoto.md
Atajos VS Code, hot reload/restart	06-cheatsheet-vscode.md
Escenarios BLoC/Supabase/GoRouter/UI + integrador	07-practicas-vscode.md
Cómo abrir DevTools, configuración	08-fundamentos-devtools.md
Widget tree, Select Widget, layout debugging	09-inspector-layout.md
Frames, flame chart, Enhance Tracing	10-performance-view.md
Sampling y flame chart de CPU	11-cpu-profiler.md
Heap, snapshots, GC, detección de leaks	12-memory-profiler.md
Requests HTTP, Supabase	13-network-view.md
Breakpoints y logpoints en DevTools	14-debugger-view.md
Niveles de package:logging, filtros	15-logging-view.md
analyze-size, límites de stores	16-app-size.md
Vista por escenario de debugging	17-cheatsheet-devtools.md
Ejercicios guiados + integrador	18-practicas-devtools.md
Modos, 60fps, herramientas de medición	19-fundamentos-rendimiento.md
const, builder, RepaintBoundary, BlocSelector	20-optimizar-rebuilds.md
Dispose, subscriptions, timers	21-memory-leak-detection.md
ListView.builder, Slivers, imágenes, CustomPainter	22-rendering-complejo.md
Tablas de diagnóstico, mitos, métricas	23-cheatsheet-optimizacion.md
6 escenarios reales + app "enferma"	24-practicas-optimizacion.md
5 bugs asíncronos, streams, isolates	25-debugging-asincrono.md
Workflow por tipo de bug + cheat sheet	26-workflow-debugging-por-tipo.md

Debugging Flutter en una línea: corre en debug para depurar, en profile para medir · pausa con breakpoints y logpoints · inspecciona con Debug Console · diagnostica con DevTools (frames, heap, requests) · optimiza rebuilds y rendering solo donde haya un problema medible · cancela subscriptions en dispose · y sigue el workflow: reproducir → aislar → diagnosticar → corregir → verificar.